

Inert Show-Cue Simulator

Assess every channel, review every cue, replay every decision

A composed teaching simulator with no external action path

Lesson	030 — capstone project	Time	240–300 minutes
Prerequisites	Lessons 028 and 029	Board	Arduino Mega 2560
Evidence	TP28, TP29, cue LEDs, RGB state, audit	Status	host verified; bench open
Energy class	E1: USB, passive switches, resistor-limited LEDs		

Boundary. This project composes synthetic switch observations with opaque visual cue labels. It has no relay, transistor-driven load, initiator, show connector, antenna, transmitter, callback, generic output sink, captured protocol, or external equipment interface. A lit cue is a teaching-state presentation only. USB power removal is the physical stop; software cancellation and shutdown are lifecycle behavior, not an emergency stop.

1 Mission and evidence chain

eight synthetic observations → channel assessment

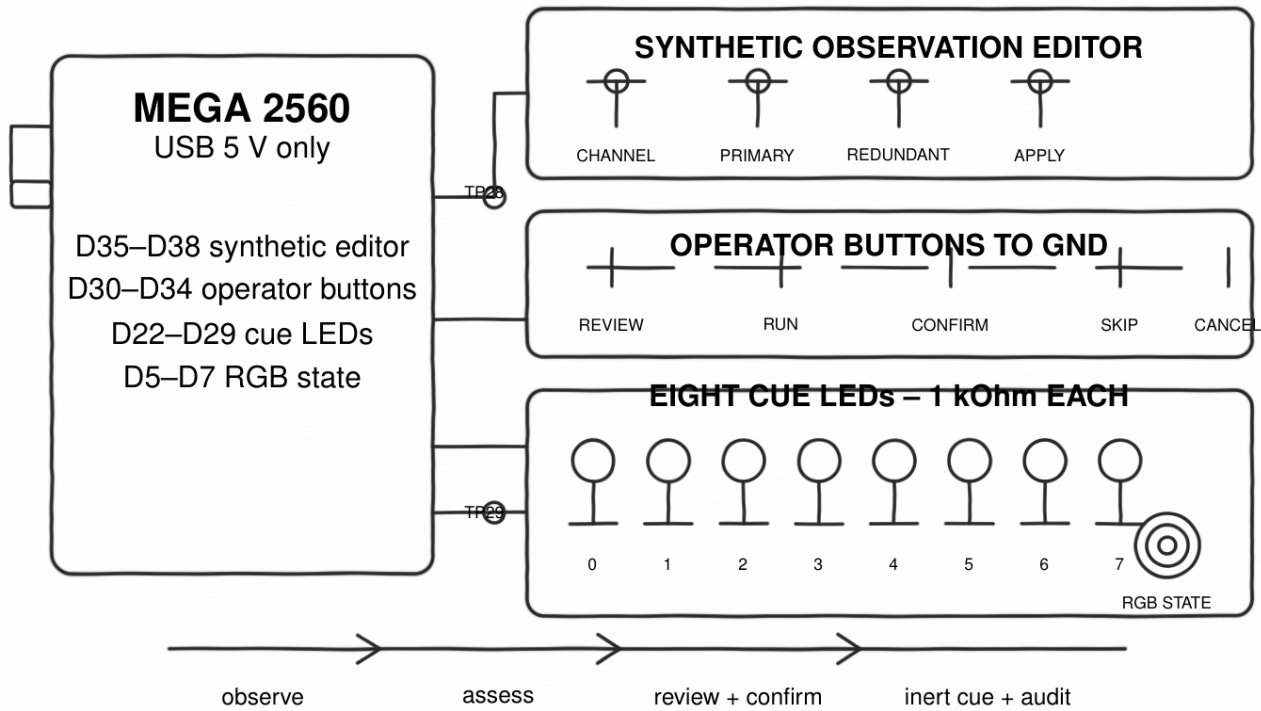
→ reviewed cue decision → inert LEDs + audit + digest.

Lesson 028 supplies bounded channel assessments. Lesson 029 supplies a copied plan, supplied-time scheduler, and append-only audit. Lesson 030 coordinates their lifecycles and decisions without owning an endpoint or performing I/O.

1.1 Learning outcomes

- compose two deterministic components through value-only contracts;
- map plan positions to channels without interpreting opaque cue IDs;
- explain Permit, Hold, and Fault evidence gates;
- apply total simultaneous-event precedence;
- reconcile snapshots, audit records, and a deterministic trace digest;
- distinguish active safe-state evidence from resource-release evidence.

2 Orientation and unpowered inspection



Alternative text: pencil orientation plate showing one USB-powered Mega, eight synthetic input switches, five pull-up operator buttons, eight separately resistor-limited cue LEDs, a separately resistor-limited common-cathode RGB state LED, TP28 at the selected synthetic input, and TP29 at the selected cue output. No external-load connection is present.

Role	Mega pin	Connection
cue indicators	D22–D29	pin to 1 kΩ, LED anode; cathode to GND
operator controls	D30–D34	Review, Run, Confirm, Skip, Cancel to GND; pull-up
synthetic editor	D35–D38	passive switch inputs used to choose test states
RGB state	D5–D7	red, green, blue through separate 1 kΩ resistors
Lesson 030 TP28	selected synthetic input	logic-level test point to GND
Lesson 030 TP29	selected cue output	Mega-pin side of resistor to GND

Unpowered prediction. Verify LED polarity, one resistor per LED channel, switch-to-ground paths, common ground, and no connection outside the USB-powered fixture.

3 Current, pins, and ownership

At most one cue channel and one RGB channel are commanded. A conservative zero-forward-voltage bound gives

$$I_{\text{channel}} \leq \frac{5.0 \text{ V}}{1000 \Omega} = 5.0 \text{ mA}, \quad I_{\text{LED, total}} \leq 10.0 \text{ mA}.$$

This is a design calculation, not a measured result.

Construction is inert. Initialization starts the assessor, then the scheduler and its audit session; failure rolls back in reverse order. Shutdown first forces the scheduler's no-visible-cue state and Shutdown record, then stops the assessor. The external audit storage deliberately remains readable.

4 Plan-to-channel composition

Cue values remain opaque visual labels. The separate map is indexed by plan position and is copied during construction.

Plan index	Cue label	Offset	Visible for	Assessed channel
0	3	2000 ms	2000 ms	5
1	7	7000 ms	2000 ms	2
2	12	12000 ms	2000 ms	7
3	29	17000 ms	2000 ms	0

Do not infer a channel from a cue. Cue 3 maps to channel 5, cue 7 maps to channel 2, cue 12 maps to channel 7, and cue 29 maps to channel 0. The cue LED is addressed with the scheduler's bounded `cueIndex`; the selected assessment uses the explicit map.

The simulator borrows the assessor, scheduler, and audit. Those objects and their caller-owned storage must outlive it. It allocates no heap storage and retains no observation pointer.

5 Complete-frame transaction

Every accepted update contains exactly eight unique channel observations. Order does not matter: the simulator canonicalizes them by channel before identity comparison and digest calculation. Null storage, a wrong count, duplicate or missing channels, invalid values, and ambiguous time are rejected.

```
const adk::InertShowInput input = {
    observations, 8, operatorInput
};
const adk::Status status = simulator.update(now, input);
const adk::InertShowSnapshot view = simulator.snapshot();
```

The pointed-to observations are consumed synchronously; no pointer survives `update()`. An identical complete frame repeated at the same timestamp is idempotent. A changed frame at that timestamp is invalid. The example samples and decides once per distinct `millis()` value. It does not submit a setup-time frame: its first update occurs only after the visible startup progression, so a same-timestamp button edge cannot be cleared by a duplicate update.

5.1 Evidence gate

Assessment	Gate	Result
Closed	Permit	confirmation may make the reviewed cue visible
Open	Hold	cue stays hidden; wait for a fresh complete frame
ShortSimulated	Hold	cue stays hidden with a distinct condition
Stale	Hold	cue stays hidden until fresh evidence arrives
Unavailable	Hold	cue stays hidden; no automatic bypass
Contradictory	Fault	latch fault and guarantee no visible cue

Recovery from a nonterminal Hold never resumes a cue automatically. The operator returns through review and supplies a fresh confirmation.

6 State and precedence

Show state	Meaning	Circuit-native evidence
Startup	coordinator not active	all cue LEDs inactive
Review	operator review gate held	review RGB pattern; cues inactive
Ready	waiting or confirming	pending-state RGB pattern; cues inactive
Running	scheduler has a visible cue	exactly one cue LED and TP29
Held	review or evidence interrupted progress	held RGB; cues inactive
Complete	plan ended	completion RGB; cues inactive
Cancelled	cancellation latched	cancelled RGB; cues inactive
Fault	contradiction, input, invariant, or audit failure	fault RGB; cues inactive

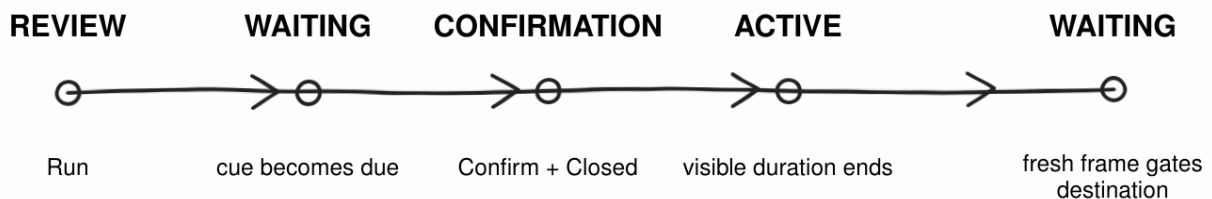
For a newly accepted full frame the total order is:

Cancel $\succ$ invalid chord $\succ$ evidence Fault/Hold $\succ$ Review release $\succ$ phase action $\succ$ time

Frame identity is checked first. Cancel therefore wins over simultaneous Confirm, Skip, or Run, but it does not turn a changed same-timestamp frame into a second accepted transaction. More than one non-cancel edge is an invalid chord. Review release prevents confirmation and immediately removes a visible cue.

7 Timing and no-catch-up rule

Run fixes the plan epoch; late decisions never move cue offsets. Confirmation is accepted only inside its inclusive window. A delayed update records and hides elapsed work, but does not briefly activate every missed cue. Advancing from one cue reaches a hidden waiting state first; a later fresh frame assesses the destination channel before requesting confirmation.



Alternative text: pencil process line from Review through Waiting, Confirmation, Active, and back to Waiting. Run starts the sequence; the cue becomes due; Confirm with a Closed assessment begins the visible interval; its end returns to a destination gate that requires a fresh frame.

At TP29 predict inactivity before accepted confirmation, a bounded active interval after it, and immediate inactivity on Hold, Cancel, Fault, completion, or shutdown. A lit LED proves only the presentation path.

8 Audit, replay, and digest

The scheduler remains the sole writer to `CueAuditBuffer`; the coordinator does not duplicate records. The ledger never overwrites old evidence and reserves room for Shutdown before a run begins. If a required transition cannot be recorded, no corresponding visible transition occurs.

The versioned trace contains supplied ticks, eight synthetic observation tokens, and debounced operator values. Replay emits stable snapshots; audit mode emits the stable Lesson 029 audit grammar. The 32-bit trace

digest detects accidental replay differences. It is neither authentication nor a security control. Repository automation replays the public fixture and checks exact snapshots and audit records behind the lesson.

8.1 Trace prediction exercise

Frame	Prediction	Observed replay	Reconciliation
Review + all Closed	_____	_____	_____
Run at first epoch	_____	_____	_____
Confirm first cue	_____	_____	_____
mapped channel Open	_____	_____	_____
Cancel + Confirm	_____	_____	_____

For one transition, copy the snapshot sequence _____, audit sequence _____, and final digest _____. Explain any disagreement: _____.

9 Narrative Mega example

```
void loop ()
{
    const adk::TimePoint now (millis ());

    observeSimulatorInput (now);
    decideInertShow       (now);
    actuateInertCues      ();
    actuateSimulatorState (now);
}
```

Objects appear in dependency order: runtime and claims, buttons, cue and state LEDs, synthetic source and assessor, plan and audit storage, scheduler, then simulator. Setup reads as acquire, configure, start. The loop reads as observe, decide, actuate. Presentation consumes one snapshot and cannot call back into the simulator. Only after every endpoint and the simulator acquire successfully does the cue row fill from left to right for two seconds. Simulator updates begin only after that dependent progression. At two minutes, an input-independent shutdown makes every output inactive. Serial replay is optional supporting evidence, never the only observation path.

10 Predict, observe, interpret

1. Before power, predict TP28 and TP29 outcomes for channel 5 Closed.
2. Start the fixture. Observe the acquisition sweep separately from the later all-cues-inactive safe state.
3. Hold Review and press Run. Predict TP29 remains inactive before confirmation even when the mapped channel is Closed.
4. Confirm cue 3. Compare the channel-5 TP28 state, selected assessment, cue-index-0 LED, TP29, and ordered audit entry.
5. Make the next mapped channel Open. Predict Hold and no cue; restore Closed, renew review, and confirm again.
6. Inject Contradictory. Predict latched Fault, inactive TP29, and no restart until reinitialization.
7. Let the fixed two-minute session expire without pressing a control. Observe immediate darkness; only then remove USB.

11 Troubleshooting tree

Observation	Stop, inspect, and interpret
more than one cue LED	remove USB; inspect bounded index presentation
wrong cue for TP28 channel	inspect the plan-position map; never use cue ID
cue active while Open/Stale	remove USB; inspect gate and fresh-frame logic
cue appears before Confirm	remove USB; reject the fixture as valid evidence
audit sequence stops	expect AuditFull and no visible transition
replay digest differs	compare exact input frames, order, and tool identity
out-of-rail TP28 or TP29	remove USB; inspect ground, switch, and resistor
hot part or unstable USB	remove USB immediately; troubleshoot unpowered

Stop rule. Remove USB power for a hot part, unstable supply, wrong polarity, missing resistor, unexpected indicator, out-of-rail TP28 or TP29, or schematic/breadboard disagreement. Do not attach an external load or real show-control connector. Software shutdown is never the physical stop.

12 Evidence status

Automated host replay, exception, Mega compilation, measured-size, PDF, and site checks remain behind this lesson. They do not constitute a physical result. Physical claims remain open until a named board, circuit, instruments, and measurements establish them; the learner is not assigned a command-line workflow or acceptance form.

13 Reference trail

- ADK `inert_channel_assessor.h`, `inert_cue_scheduler.h`, `inert_show_simulator.h`, focused tests, replay fixture, and canonical Lesson 030 sketch;
- ADK development, testing, safety, packaging, and PDF policies;
- Lessons 028 and 029 for dependency contracts and evidence limits;
- Arduino Mega 2560 Rev3 datasheet for the exact board profile.

The searchable HTML reference is the primary accessible route. This printable PDF uses real text, headings, lists, and monochrome graphics but does not claim PDF/UA conformance.